

EPPS6323 Knowledge Mining

Name: Ann Nibana SL

EPPS:6323

Assignment 4

Text Analytics using quanteda

1. Read tutorial of quanteda at <https://tutorials.quanteda.io/>
2. Download quanteda_textanalytics02.R from Teams
3. Analyze:
 - a. Biden-Xi summit data

The document made use of here is the Sunnylands Statement on Enhancing Cooperation to Address the Climate Crisis, a joint U.S.-China statement issued on November 14, 2023, following the Biden-Xi summit in San Francisco.

In **Step 1**, the code extracts text from the PDF file xi-biden.pdf (located at "C:/Users/annni/Documents/1_Spring_2025/1_EPPS6323/Assign_4/Lab03/xi-biden.pdf") using pdf_text(). The extracted text, which has span multiple pages, is combined into a single string using paste() with a space separator.

The combined text is then saved as a .txt file (xi-biden.txt) in the same directory for easier handling.

Finally, a data frame (summit_data) is created with the text and metadata, including a document ID (xi-biden) and a flag (stringsAsFactors = FALSE) to keep as character vectors.

```
# Load required libraries
library(pdftools)
library(quanteda)
library(quanteda.textstats)
library(quanteda.textplots)

# Step 1: Extract text from the PDF
pdf_path <- "C:/Users/annni/Documents/1_Spring_2025/1_EPPS6323/Assign_4/Lab03/xi-biden.pdf"
text <- pdf_text(pdf_path)

# Combine the text into a single string (if the PDF has multiple pages)
text_combined <- paste(text, collapse = " ")

# Save the extracted text as a .txt file for easier handling
writeLines(text_combined, "C:/Users/annni/Documents/1_Spring_2025/1_EPPS6323/Assign_4/Lab03/xi-biden.txt")

# Create a data frame with the text and metadata
summit_data <- data.frame(
  text = text_combined,
  doc_id = "xi-biden",
  stringsAsFactors = FALSE
)
```

To help keep track of the document's context labeling the document as "Xi-Biden Summit," setting the year to 2023 is done.

Then, in **Step 2**, it creates a corpus (a structured collection of text) from the data, telling Quanteda which column contains the text (text) and which column identifies the document (doc_id).

Finally, it preprocesses the text by breaking it into words (tokenizing), removing punctuation, numbers, and common words like "the" or "and" (stopwords), and converting everything to lowercase.

```
# Add document variables (docvars) for context
summit_data$Document <- "Xi-Biden Summit"
summit_data$Year <- 2023 # Assuming this is from the 2023 summit; adjust if needed
summit_data$Summit <- "Biden-Xi"

# Step 2: Create a corpus
summit_corpus <- corpus(summit_data, text_field = "text", docid_field = "doc_id")
summary(summit_corpus)

# Preprocess the text: tokenize, remove punctuation, numbers, stopwords, and convert to lowercase
summit_tokens <- tokens(summit_corpus,
                        remove_punct = TRUE,
                        remove_numbers = TRUE) %>%
  tokens_remove(stopwords("english")) %>%
  tokens_tolower()
```

In **Step 3**, it performs a frequency analysis by finding the top 10 most frequent words in the trimmed DFM using `textstat_frequency()` and prints the results. This step helps identify the most common words in the document, revealing key themes or topics, such as climate or cooperation, in the context of the Biden-Xi summit.

```
# Create a document-feature matrix (DFM)
summit_dfm <- dfm(summit_tokens)

# Trim the DFM to remove rare terms (optional, for cleaner analysis)
summit_dfm_trim <- dfm_trim(summit_dfm, min_termfreq = 2)

# Step 3: Frequency Analysis
# Get the top 10 most frequent terms
top_features <- textstat_frequency(summit_dfm_trim, n = 10)
print(top_features)

> # Get the top 10 most frequent terms
> top_features <- textstat_frequency(summit_dfm_trim, n = 10)
> print(top_features)
  feature frequency rank docfreq group
1  countries      19    1      1  all
2  including      19    1      1  all
3   china       18    3      1  all
4   states      17    4      1  all
5   climate      16    5      1  all
6   united      15    6      1  all
7   paris       14    7      1  all
8 cooperation      13    8      1  all
9 agreement      12    9      1  all
10      two       10   10      1  all
```

From the result it can be seen that the Biden-Xi summit prioritized climate cooperation, with a strong focus on U.S.-China collaboration within the framework of the Paris Agreement.

The document emphasizes multilateral engagement (“countries,” “including”) and bilateral partnership (“china,” “united states,” “two”), framing climate action (“climate,” “paris,”

“agreement”) as a shared global responsibility. The term “cooperation” underscores the summit’s diplomatic approach, aiming to strengthen U.S.-China ties on climate issues.

- b. US presidential inaugural speeches
 - i. Any similarities and differences over time and among presidents?

As part of the text analysis workflow for presidential inaugural speeches, I decided to focus on comparing conservative and liberal presidents. In order to achieve that libraries used (quanteda.textmodels, quanteda.textstats, ggplot2, tidyr) for text processing and visualization.

It then defines two groups: conservative presidents (Reagan, Trump, Bush) and liberal presidents (Kennedy, Clinton, Obama). A subset of the inaugural speech corpus is created, including only these presidents. Then the code processes the corpus by removing punctuation, English stopwords, and tokens shorter than 5 characters, also trims terms appearing in fewer than 5 documents.

Finally, it calculates the relative frequency of terms in the processed corpus, weighted by their document frequency (prop scheme), grouped by president, to analyze word usage differences among presidents.

```
# Load required libraries
library(quanteda)
library(quanteda.textmodels)
library(quanteda.textstats)
library(quanteda.textplots)
library(ggplot2)
library(tidyr)

# Define conservative and liberal presidents
conservative_pres <- c("Reagan", "Trump", "Bush")
liberal_pres <- c("Kennedy", "Clinton", "Obama")

# Subset inaugural corpus for selected presidents
subset_corpus <- corpus_subset(data_corpus_inaugural,
                               President %in% c(conservative_pres, liberal_pres))

# --- STEP 1: Relative frequency ---
dfm_subset <- subset_corpus %>%
  tokens(remove_punct = TRUE) %>%
  tokens_remove(stopwords("english")) %>%
  dfm() %>% # <-- THIS was missing!
  dfm_group(groups = President) %>%
  dfm_trim(min_termfreq = 5)

# Compute relative frequency
dfm_rel <- dfm_weight(dfm_subset, scheme = "prop") * 100
rel_freq <- textstat_frequency(dfm_rel, groups = dfm_rel@docvars$President)
```

To Plot

In **Step 1**, it filters the top 30 terms per president based on their relative frequency, grouping by president and ungrouping afterward to prepare the data for visualization.

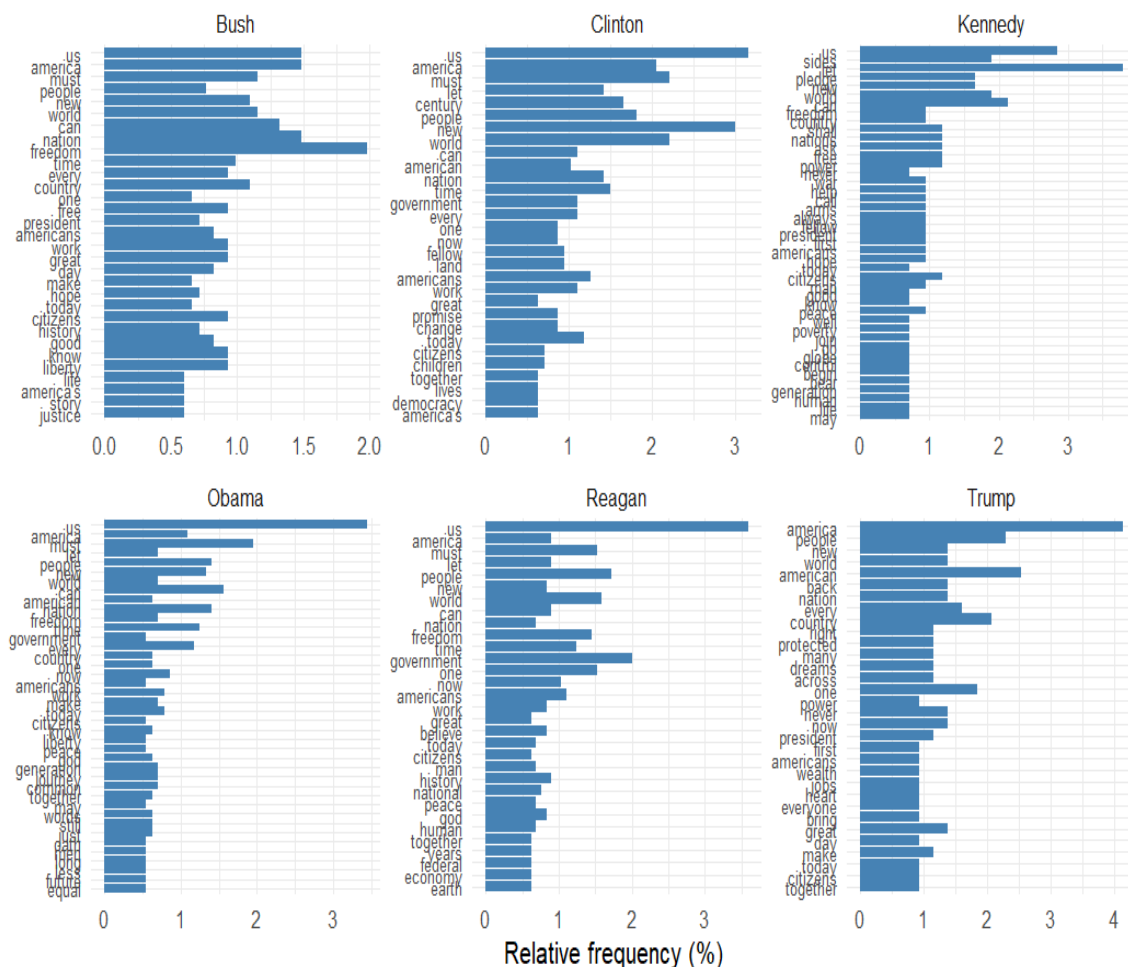
Step 2 creates a label for the x-axis by combining each term (top_rel_freq\$feature) with its relative frequency (top_rel_freq\$frequency), separated by a space, to ensure unique labels.

Step 3 uses ggplot2 to create a plot, mapping the term labels to the x-axis, relative frequency to the y-axis, and coloring by president (group). The plot is a column chart (geom_col) with free scales for the x-axis and customized labels for the axes and title ("Top 10 Inaugural Terms by Relative Frequency (per President)"). The text size is set to 12 for the base theme and 8 for axis labels, providing a clear visualization of the most frequent terms used by each president.

```
# Step 1: Filter top 30 terms per president
top_rel_freq <- rel_freq %>%
  group_by(group) %>%
  slice_max(order_by = frequency, n = 30) %>%
  ungroup()

# Step 2: Create a better label for x-axis to keep it unique across facets
#top_rel_freq$label <- paste(top_rel_freq$feature, top_rel_freq$group, sep = "_")
top_rel_freq$label <- top_rel_freq$feature
# Step 3: Plot
ggplot(top_rel_freq, aes(x = reorder(label, frequency), y = frequency)) +
  geom_col(fill = "steelblue") +
  facet_wrap(~ group, scales = "free") +
  coord_flip() +
  labs(x = NULL, y = "Relative frequency (%)") +
  ggtitle("Top 10 Inaugural Terms by Relative Frequency (per President)") +
  theme_minimal(base_size = 12) +
  theme(axis.text.y = element_text(size = 8))
```

Top 10 Inaugural Terms by Relative Frequency (per President)



```

> # Step 4: Print unique top words per president
> unique_terms %>%
+   group_by(group) %>%
+   summarise(unique_words = paste(label, collapse = ", ")) %>%
+   print()
# A tibble: 6 x 2
  group    unique_words
  <chr>    <chr>
1 Bush    justice, story
2 Clinton century, land, change, promise, children, lives, democracy
3 Kennedy sides, pledge, shall, nations, ask, war, always, help, arms, call, well, poverty, globe, go, bear, control...
4 Obama   common, journey, still, words, just, oath, men, long, less, future, equal
5 Reagan  believe, national, earth, economy, federal, years
6 Trump   back, across, right, many, dreams, protected, bring, jobs, wealth, heart, everyone

```

Over the years, U.S. presidential inaugural speeches have shown a clear shift in language that reflects evolving national priorities and communication styles.

Earlier speeches, like president Kennedy's, emphasized Cold War-era ideals such as **freedom, pledge, and liberty** reflecting a focus on global leadership and civic duty. After 9/11, Bush's speeches emphasized **freedom, justice, and security** capturing the post-terrorism climate and America's reasserted global stance.

As time progressed, particularly from the 1990s onward, there was a noticeable rise in optimistic, future-oriented language in speeches by Clinton and Obama, with terms like **change, generation, and hope** signaling a focus on domestic progress and inclusivity.

More recently, Trump's inaugural address reflected a shift toward populism and nationalism, using words like **America, power, and jobs**, which underline economic self-reliance and a focus on national identity.

There is a clear evolution in political language from abstract ideals (freedom, liberty) to more tangible, issue-based language (jobs, economy, healthcare).

The shift reflects changing voter concerns, global events, and communication styles across generations. It is seen that Language is becoming more personalized, simplified, and emotionally driven in recent speeches.

What is Wordfish?

Word fish is an Unsupervised text scaling method that involves figuring out how different texts are, by looking at which words they use often and placing them on a meaningful scale without any prior labeling. It is based on a Poisson equation that focuses on β_w , the **word discrimination parameter**, which reflects how strongly a word distinguishes between ideological positions; and θ_d **document's position on the latent scale**, the output of interest.

For instance if using Political documents, You don't tell the wordfish algorithm (running inside `quanteda.textmodels`) what's left-wing or right-wing. You don't provide categories like "liberal" or "conservative." The model figures out the differences between texts on its own.

Other Uses:

Analyzing political speeches (e.g .Trump vs. Obama)

Estimating policy position shifts over time.
Comparing government vs. opposition texts.
Understanding media bias.

4. How to compare positions (hint: wordfish and scaling methods)

In Wordfish, comparing positions refers to identifying where different documents or speakers lie on an underlying ideological or thematic scale, based on the frequency of the words they use. This is done through a statistical model where each document receives a position score (called **theta**, θ) that reflects its stance like left vs right, liberal vs conservative, etc.

Below shows a code which performs a Wordfish analysis using the `quanteda.textmodels` package to estimate the ideological positions (theta values) of documents in the built-in dataset **data_corpus_irishbudget2010**.

```
library(quanteda)
library(quanteda.textmodels)

# Use built-in dataset
data(data_corpus_irishbudget2010)

# To see how many documents in this built in dataset
ndoc(ie_dfm)

# Step 1: Convert to DFM
ie_dfm <- dfm(tokens(data_corpus_irishbudget2010))

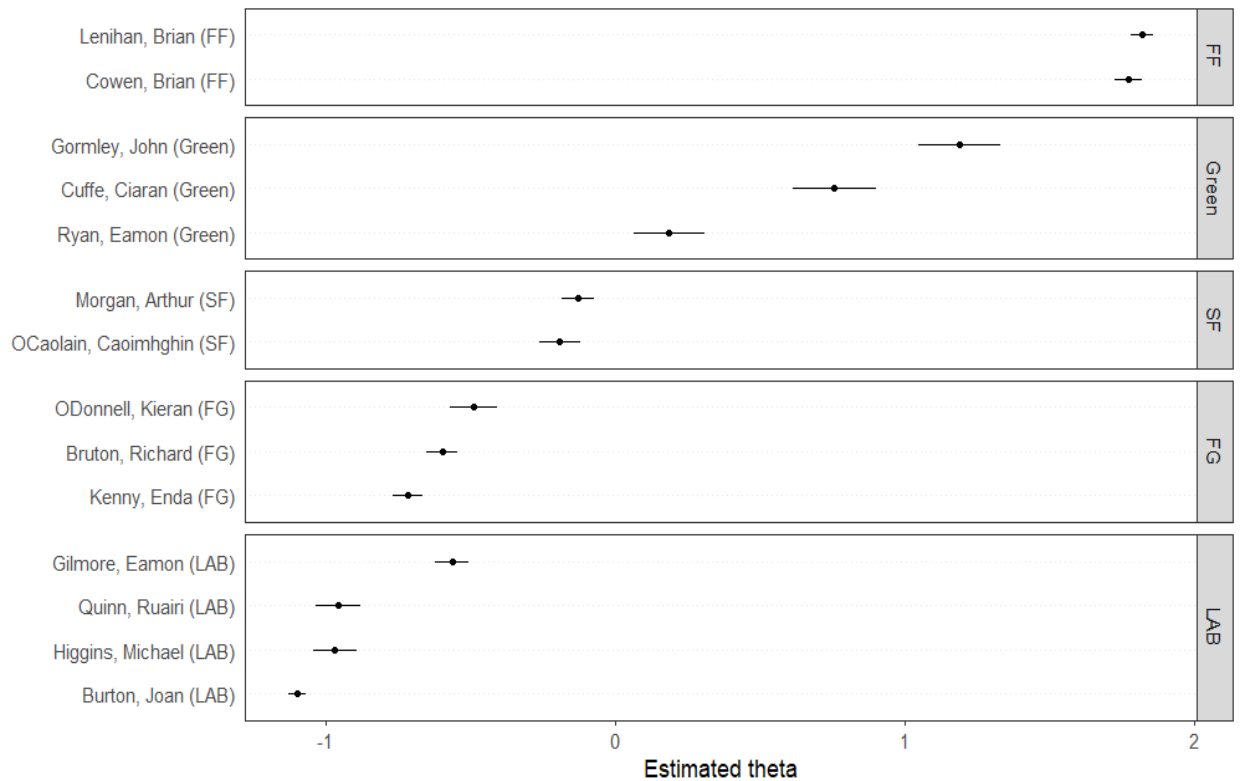
# Step 2: Run Wordfish
wf <- textmodel_wordfish(ie_dfm, dir = c(6, 5)) # anchor two documents for direction

# Step 3: Plot document positions
textplot_scale1d(wf, margin = "documents",
                 groups = data_corpus_irishbudget2010$party)
```

The `Corpus` contains speeches from Irish budget debates. It is converted into a **document-feature matrix (DFM)** using `tokens()` and `dfm()`, which prepares the text for modeling by counting word frequencies across documents.

The `textmodel_wordfish()` function estimates the positions (**theta**) for each document. The `dir = c(6, 5)` argument anchors the scale using documents 6 and 5 to define the direction of the ideological axis (e.g., liberal to conservative).

`textplot_scale1d()` visualizes the **theta scores** (document positions) along a one-dimensional axis, grouped by political party.



This plot shows the estimated ideological positions (**theta values**) of various Irish politicians from different political parties based on their speeches.

The horizontal axis represents the **latent scale**, with each point indicating where a politician's speech falls on that scale.

Politicians from party **Fianna Fail (FF)**, such as Brian Lenihan and Brian Cowen, are placed at the **far right**, suggesting their language aligns with more conservative or pro-government stances.

Politicians from the **Labour Party (LAB)** like Joan Burton and Michael Higgins are positioned **far left**, indicating more progressive or opposition-oriented rhetoric.

Fine Gael (FG) members cluster toward the center-left, while **Green Party** members occupy a **moderate-right position**, and **(SF)** lies somewhere in between.

5. Create a corpus using government documents selected from the govinfo.gov website (use sample program govtdata01.R)

Step1,2:

Create CSV file with links to the pdfs based on 119TH CONGRESS OVERSIGHT AND AUTHORIZATION PLAN. Here I have chosen to download and analyze 3 pdfs.

```
# Load necessary libraries
library(quanteda)
library(quanteda.textmodels)
library(readr)
library(purrr)

# Step 1: Read the CSV file with document info
data <- read_csv("C://Users//annni//Documents//1_Spring_2025//1_EPPS6323//Assign_4//Lab03//congresshearing119doc.csv")

# Step 2: Check structure (optional)
spec(data) #Shows column types like Title, PDF link, Date
```

The process begins by importing necessary R libraries (mainly quanteda and its extensions , readr, purrr).

A CSV file is read which contains links to congressional hearing documents (in PDF format) along with their metadata (title and date).

The structure of the file is inspected using spec() to confirm it includes expected columns such as PDF link.

Step3,4,5,6 :

Set up a folder to download necessary pdf files

Using the download.file() function wrapped inside a custom function download_govfiles_pdf(), the code downloads multiple PDFs from the GovInfo website.

The files are saved in a local folder named gov_pdfs with randomized delays (Sys.sleep(runif())) between downloads to avoid server overload.


```

#Step 3: Prepare download info
pdf_links <- data$`PDF link`
pdf_ids <- seq_along(pdf_links) # Or use titles if needed

# Step 4: Set folder to save PDFs (Update to your folder!)
save_dir <- "C://Users//annni//Documents//1_Spring_2025//1_EPPS6323//Assign_4//Lab03//gov_pdfs/"

# Step 5: Function to download PDFs safely
download_govfiles_pdf <- function(url, id) {
  tryCatch({
    destfile <- paste0(save_dir, "govfile_", id, ".pdf")
    download.file(url, destfile = destfile, mode = "wb")
    Sys.sleep(runif(1, 1, 3)) # Avoid server overload
    return(paste("Downloaded:", destfile))
  }, error = function(e) {
    return(paste("Failed:", url))
  })
}

```

Step 7

Text Extraction and Corpus Creation

- The downloaded PDFs are loaded using `pdftools::pdf_text()` which extracts the full text from each file.
- Each document's text is stored as a string, and all texts are combined into a corpus using `quanteda::corpus()` for further analysis.

```

# Step 7: Load PDF files create corpus, Tokenize and clean
library(pdftools)
pdf_files <- list.files(save_dir, pattern = "\\*.pdf$", full.names = TRUE) # Looks inside your gov_pdfs

#Extract text from Pdf
pdf_texts <- lapply(pdf_files, pdf_text)
pdf_texts <- sapply(pdf_texts, paste, collapse = " ") # Combine all pages per PDF into one document

#create corpus
gov_corpus <- corpus(pdf_texts)
docnames(gov_corpus) <- basename(pdf_files) # Optional: name docs by file name
summary(gov_corpus) # Tokens:total no of words, Values:unique words if present,sentences:no of senten

```

Tokenize, clean, plot

The raw text is cleaned by:

- Removing punctuation and symbols.
- Converting all words to lowercase.
- Removing English stopwords (the, and, is)

Tokenized words are then stored in a Document-Feature Matrix (DFM), which counts how many times each word appears in each document.

Tokenize,clean,plot

```
# Tokenize and clean text
gov_tokens <- tokens(gov_corpus,
                     remove_punct = TRUE,
                     remove_symbols = TRUE) %>%
  tokens_tolower() %>%
  tokens_remove(stopwords("english")) %>%
  tokens_keep(pattern = "[a-z]{3,}$", valuetype = "regex")

# Create a Document-Feature Matrix
gov_dfm <- dfm(gov_tokens)

# (Optional): Trim low frequency terms
gov_dfm <- dfm_trim(gov_dfm, min_termfreq = 5)
```

Step 8

From the DFM, the top 20 most frequent words in each document are computed using `textstat_frequency()` from `quanteda.textstats`.

A bar chart is plotted using `ggplot2` where each panel represents one document, showing its top 20 words sorted by frequency.

```
#step8: compare text and plot
# Top words per document
library(quanteda.textstats)
#textstat_frequency(gov_dfm, groups = docnames(gov_dfm))
# Get top 20 words per document
top20_words <- textstat_frequency(gov_dfm, groups = docnames(gov_dfm), n = 20)
# View results
head(top20_words, 60) |
```

Plot

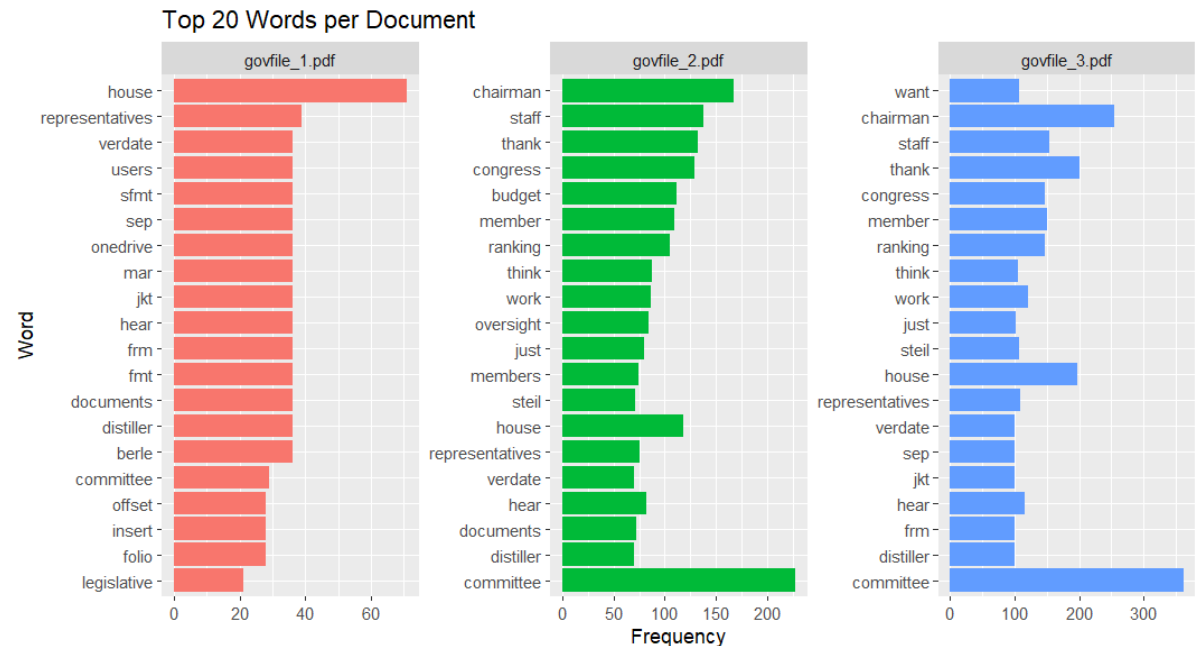
Using `ggplot2` and `dplyr`, the words were first grouped by document and then reordered based on how frequently they appeared within each document.

The plot uses separate panels (facets) for each document, making it easy to compare dominant terms across files like `govfile_1.pdf`, `govfile_2.pdf`, and `govfile_3.pdf`

```
library(ggplot2)
library(dplyr)

# Create a new column that reorders words within each group
top20_words <- top20_words %>%
  group_by(group) %>%
  mutate(feature_ordered = reorder(feature, frequency))

# Now plot using this reordered feature
ggplot(top20_words, aes(x = feature_ordered, y = frequency, fill = group)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ group, scales = "free") +
  coord_flip() +
  labs(x = "Word", y = "Frequency", title = "Top 20 Words per Document")
```



In **govfile_1.pdf**, terms like "house," "representatives," "verdate," and "budget" dominate, suggesting a focus on legislative structure, budgeting, and procedural documentation, likely outlining the House's oversight framework.

In **govfile_2.pdf**, words such as "chairman," "staff," "congress," and "budget" are prominent, indicating an emphasis on leadership roles, congressional operations, and financial oversight, possibly detailing committee responsibilities.

Finally, **govfile_3.pdf** highlights "want," "chairman," "staff," and "congress," reflecting a similar focus on leadership and operations but with a potential emphasis on priorities or requests ("want"). Common terms across all documents, like "house," "congress," "budget," and "committee," underscore a consistent theme of legislative oversight, financial planning, and committee activities.